Annexe — Se connecter à PostgreSQL (poste local ou en ligne)

Table of Contents

1. Paramètres de connexion (à retenir) . . . 1
2. Option A — Conteneur PostgreSQL dédié (recommandé) . . . 2
   2.1. A.1 Démarrer / arrêter . . . 2
   2.2. A.2 Dans le terminal (psql intégré au conteneur) . . . 2
   2.3. A.3 (Re)charger un script de données . . . 2
   2.4. A.4 Depuis Visual Studio Code . . . 3
   2.5. A.5 Depuis DBeaver (installé sur les postes) . . . 3
3. Option B — Neon (PostgreSQL en ligne) . . . 3
4. Option C — Autres outils (au choix) . . . 4
5. Éviter les conflits entre conteneurs (important si vous débutez avec Docker) . . . 5

Comment **exécuter le SQL du module** : (A) un conteneur PostgreSQL **dédié** sur le poste (terminal ou VS Code), (B) le service en ligne **Neon**, © d'autres outils. Toutes les requêtes du cours, des TP et des DST fonctionnent à l'identique sur l'une ou l'autre de ces cibles.

1. Paramètres de connexion (à retenir)

Quelle que soit la méthode locale, la base de ce module utilise **toujours** les mêmes paramètres :

Paramètre	**Valeur**
Hôte	`localhost` (ou `127.0.0.1`)
Port	**5440**
Base de données	`sql_cours`
Utilisateur	`prof_sio`
Mot de passe	`prof_sio`



Le port **5440** (au lieu du 5432 habituel) est volontaire : il évite tout conflit avec d'autres PostgreSQL déjà lancés sur le poste (voir la section « Éviter les conflits »).



Outils disponibles en salle (SLAM) : DBeaver est **installé** (client graphique, cf. A.5) et **Visual Studio Code** est disponible le cas échéant (cf. A.4). **pgAdmin n'est pas installé** sur les postes. Le plus rapide sans rien configurer reste le **terminal dans le conteneur** (A.2).

2. Option A — Conteneur PostgreSQL dédié (recommandé)

Une pile Docker **isolée** est fournie à chaque séance dans le fichier compose.yml dans `compose.yml`. Elle crée un PostgreSQL 16 **rien que pour ce module** (nom, port et volume propres).

2.1. A.1 Démarrer / arrêter

Depuis le répertoire dans lequel vous avez placé le TP du jour avec le fichier compose.yml :

```
# démarrer (en arrière-plan)
docker compose up -d

# arrêter (les données sont conservées)
docker compose down

# tout remettre à zéro ( efface les données du volume)
docker compose down -v
```

Au **tout premier démarrage**, les jeux de données du cours (`cinema`, `football`, `ecole`, `geo`) sont **chargés automatiquement** (dossier `scripts/` dans le répertoire du jour monté dans `docker-entrypoint-initdb.d`). Rien d'autre à faire pour commencer.

2.2. A.2 Dans le terminal (psql intégré au conteneur)

Le plus simple : ouvrir `psql` **à l'intérieur** du conteneur (aucune installation sur le poste) :

```
docker exec -it sql_cours_db psql -U prof_sio -d sql_cours
```

Puis, dans l'invite `sql_cours=#` :

```
SET search_path TO cinema;   -- se placer sur un jeu de données
\dt                          -- lister les tables
SELECT titre, annee FROM film ORDER BY annee LIMIT 5;
\q                           -- quitter
```

Commandes `psql` utiles : `\l` (bases), `\dn` (schémas), `\dt` (tables), `\d film` (structure d'une table), `\x` (affichage étendu), `\q` (quitter).

2.3. A.3 (Re)charger un script de données

Les scripts sont **rejouables**. Méthode fiable sous Windows (préserve l'UTF-8) : on **copie** le fichier dans le conteneur puis on l'exécute avec `-f`.

```
docker cp .\scripts\cinema.sql sql_cours_db:/tmp/cinema.sql
docker exec sql_cours_db psql -U prof_sio -d sql_cours -f /tmp/cinema.sql
```



Évitez `Get-Content script.sql | docker exec … psql` sous PowerShell : cela peut **corrompre les accents** (encodage). Préférez `docker cp` + `-f` comme ci-dessus.

2.4. A.4 Depuis Visual Studio Code

Oui, c'est possible et confortable (éditeur + exécution + grille de résultats) :

1. Installer l'extension « **PostgreSQL** » de Microsoft (identifiant `ms-ossdata.vscode-pgsql`). *Alternative éprouvée :* **SQLTools** + le pilote « **SQLTools PostgreSQL/Redshift Driver** ».
2. **Ajouter une connexion** avec les paramètres du tableau ci-dessus (hôte `localhost`, port `5440`, base `sql_cours`, utilisateur `prof_sio`, mot de passe `prof_sio`).
3. Ouvrir un fichier `.sql` (par exemple un corrigé), sélectionner une requête et l'**exécuter** (`Ctrl+Entrée` ou clic droit → *Run*) : le résultat s'affiche dans une grille.



Pour interroger un jeu précis sans le préfixer partout, exécutez d'abord `SET search_path TO cinema;` dans la même session/onglet SQL.

2.5. A.5 Depuis DBeaver (installé sur les postes)

DBeaver est un client graphique gratuit, déjà **installé sur les postes de la salle**.

connexion manuelle :

1. Menu *Base de données* → *Nouvelle connexion* → ***PostgreSQL***.
2. Saisir les paramètres du tableau ci-dessus : Hôte `localhost`, Port `5440`, Base `sql_cours`, Utilisateur `prof_sio`, Mot de passe `prof_sio` (cocher « Enregistrer le mot de passe »).
3. *Tester la connexion* — la 1re fois, DBeaver propose de **télécharger le pilote PostgreSQL** : accepter — puis *Terminer*.
4. Ouvrir un **éditeur SQL** (icône **SQL** ou `Ctrl+]`), écrire une requête et l'exécuter avec `Ctrl+Entrée` : les résultats s'affichent dans une grille.



Dans l'arborescence de gauche : **sql_cours** → **Schemas** → **cinema** / **football** / **ecole** / **geo** → **Tables**. Un double-clic sur une table montre ses données. En tête d'un script, pensez à `SET search_path TO cinema;`.

3. Option B — Neon (PostgreSQL en ligne)

Neon (https://neon.tech) héberge un PostgreSQL gratuit dans le cloud. Idéal si l'on ne veut **rien installer** ou travailler depuis n'importe quel poste.

1. Créer un compte, puis un **projet** (choisir une **région Europe**).
2. Neon fournit une **chaîne de connexion** du type :

   ```
   postgresql://prof_sio:motdepasse@ep-xxxx-123456.eu-central-
   1.aws.neon.tech/sql_cours?sslmode=require
   ```

   Le `sslmode=require` est obligatoire (connexion chiffrée).
3. **Éditeur SQL web** (aucune installation) : dans le tableau de bord Neon, ouvrir *SQL Editor*, **coller** le contenu d'un script (`cours-adoc/scripts/cinema.sql`, etc.), puis *Run*.
4. **Préparer la base pour les TP en une seule fois** : coller `cours-adoc/scripts/setup-neon.sql` dans le *SQL Editor* puis *Run* — cela crée d'un coup les 4 schémas (`cinema`, `football`, `ecole`, `geo`). Script **autonome** (pas de `\include`) et **rejouable**. □□ à exécuter **en un seul Run** (chaque section fixe son propre `search_path`).
5. Ou se connecter avec **VS Code** / **DBeaver** en utilisant la chaîne de connexion (SSL activé).

Table 1. Local vs Neon

Neon (en ligne)	**Conteneur local**
Rien à installer, accessible partout	Fonctionne **hors ligne**
Nécessite un compte + Internet	Aucune donnée envoyée sur Internet
Données hébergées dans le cloud	Données sur le poste



Sur Neon (cloud), **n'importez jamais de données personnelles réelles** (copies d'élèves, noms, notes nominatives) — uniquement les jeux de données pédagogiques du cours.

4. Option C — Autres outils (au choix)

Tous se connectent avec les **mêmes paramètres** (Option A) ou la **chaîne Neon** (Option B) :

- **DataGrip / l'outil « Database » intégré aux IDE JetBrains** (IntelliJ, PhpStorm…) : si vous utilisez déjà un IDE JetBrains, ajoutez une *Data Source → PostgreSQL*.
- **Adminer** : interface web ultra-légère (un seul fichier / un petit conteneur), pratique en salle.
- **psql installé sur le poste** : si un jour le client `psql` est installé (installeur PostgreSQL, ou `winget install PostgreSQL.PostgreSQL`), on peut se connecter directement : `psql -h localhost -p 5440 -U prof_sio -d sql_cours`.



pgAdmin n'est **pas installé** sur les postes de la salle : privilégier **DBeaver** (A.5) ou **VS Code** (A.4). Il resterait utilisable sur un poste personnel si besoin.

5. Éviter les conflits entre conteneurs (important si vous débutez avec Docker)

Sur un même poste, **deux conteneurs ne peuvent pas exposer le même port hôte**. Si un autre PostgreSQL occupe déjà `5432` (module PHP, Bloc2, Bloc3…), le démarrer une 2e fois sur `5432` échoue avec `port is already allocated`. La pile de ce module évite le problème car elle est **entièrement isolée** :

Élément	Valeur (unique à ce module)
Projet Compose (`name`)	`sql-cours`
Conteneur	`sql_cours_db`
Port hôte	`5440`
Volume de données	`sql_cours_pgdata`

Bonnes pratiques :

- **Un port hôte différent par pile** (ex. 5432 pour l'un, 5433 pour un autre, **5440** ici).
- **Un nom de conteneur et un volume distincts** par projet (déjà le cas ici).
- Voir ce qui tourne et sur quels ports :

```
docker ps                 # conteneurs actifs : NAMES et PORTS
docker compose ls         # piles Compose lancées
```

- Libérer un port en **arrêtant** la pile qui l'utilise (sans supprimer ses données) :

```
docker compose -f <chemin-de-son-compose.yml> down
# ou, pour un conteneur isolé :
docker stop <nom_du_conteneur>
```

- Changer le port de **ce** module si `5440` est déjà pris : modifier `"5440:5432"` dans `docker/compose.yml` (par ex. `"5441:5432"`) et adapter le port dans vos outils.



Cette pile ne touche **pas** aux autres conteneurs : la lancer ou la supprimer n'affecte ni le module PHP, ni les autres blocs.